A novel initialization method for solving flexible job-shop scheduling problem

Shi Yang[1], Zhang Guohui[1], Gao Liang[1*], Yuan Kun[2]

[1] The State Key Laboratory of Digital Manufacturing Equipment and Technology, Huazhong University of Science and Technology, 430074, China (gaoliang@mail.hust.edu.cn)
[2] ENIM, Ile du Saulcy, F57045 Metz Cedex 1

ABSTRACT

A novel initialization method was proposed for Genetic Algorithm (GA) to generate high-quality initialization population so as to solve Flexible Job-shop Scheduling Problem (FJSP). The novel initialization method consists of two sub-methods: Global Selection (GS) and Local Selection (LS). GS is used to find different initial assignments in different runs of the algorithm, and to enhance the capability of exploring search space considering the workload of all machines, while LS can find the shortest occupation time machine in alternative machine set of each job. To prove the efficiency of this initialization method, various benchmark data taken from the literature are tested, and the computation results show that this method can shorten the computational time and generate better results.
Keywords: Genetic algorithm, Flexible job shop scheduling, Initialization

1. INTRODUCTION

Scheduling is one of the most important issues in the planning and operation of manufacturing systems [1], and in the real manufacturing systems, an operation could be allocated to a suitable machine from a set of alternative machines and these machines are duplicated to balance the workload. It is known as the Flexible Job-shop Scheduling Problem (FJSP).

FJSP contains two sub-problems: the routing sub-problem which is assigning each operation to a machine selected out of a set of capable machines and the scheduling sub-problem, which consists of sequencing the assigned operations on all machines in order to obtain a feasible schedule to minimize the predefined objective function.

In recent years, the adoption of meta-heuristics such as tabu search (TS), simulated annealing (SA), particle swarm optimization (PSO), genetic algorithm (GA) has led to better results than the traditional dispatching or greedy heuristic algorithm [2-5]. Meta-heuristic methods produce reasonable good schedules in a reasonable computational time, and could get near optimal solutions easily.

Population initialization is a crucial task in evolutionary algorithms because it can affect the convergence speed and also the quality of the final solution [6]. Kang et al. [7] proposed an improved initialization method based on genetic algorithm for solving the traveling salesman problem (TSP). The computational results proved that the method solved TSP more effectively. But the method was difficultly used to solve the FJSP. Kacem et al. [8] proposed the approach by localization (AL) and an evolutionary approach to initialize population, and it performed well in solving FJSP.

978-1-4244-4136-5/09/$25.00 ©2009 IEEE

The research on initialization method is limited, so essential research on the aspect is still needed. In this paper, we present a novel initialization method integrating different strategies to generate high-quality initial population in order to shorten the computational time and enhance the efficiency of getting the optimal solution (or near optimal solution). The method contains two parts: Global Selection and Local Selection which could generate good initialization population.

2. PROBLEM DESCRIPTION

The flexible job-shop scheduling problem could be described as follows. There are N jobs and M machines, the problem is how to allocate the jobs to machines to satisfy some criteria. We denote a set of N jobs $J = \{J_1, J_2,...,J_i,...,J_N\}$ and M machines $M= \{M_1, M_2,...,M_k,...,M_M\}$ here. Each job J_i has the given number non-preemptable operations in a predetermined sequence, each operation j of job J_i (denoted O_{ij}) requires one machine selected from a set of available machines, and O_{ij} will take an occupation of the selected machine from the set starting time to the set ending time. In this paper, the objective function is to minimize the maximal completion time (makespan) of all jobs.

Hypotheses considered in this paper are summarized as follows:

1) All machines are available at time 0;
2) All jobs are released at time 0;
3) Each machine can process only one operation at a time;
4) Each operation can be processed without interruption on one of a set of available machines;
5) Recirculation occurs when a job could visit a machine more than once;

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 22,2026 at 14:13:38 UTC from IEEE Xplore. Restrictions apply.

6) The order of operations for each job is predefined and cannot be modified.

An instance of FJSP is given in Table 1 so as to describe the method clearly and easily. In table 1, as the definitions above, we can easily know that there are 3 jobs and 5 machines, J_1 has 2 operations, J_2 has 3 operations and J_3 has 2 operations, each cell denotes the occupation time of that operation on the corresponding machine, and "-" denotes that the operation cannot be executed by the corresponding machine, the problem like this instance is a partial flexibility job-shop scheduling problem [8], actually partial flexibility job shop scheduling problem is the extension of total flexibility job-shop scheduling problem, because we can consider the occupation time of the unavailable machines as infinity, then the problem will be changed into a special total flexibility problem, but its search space will be more complicated and the computation time will increase, so it makes the problem more difficult, and we take this problem as an instance to make a more representative presentation.

Table 1 An instance of FJSP

Job	*Operation*	M_1	M_2	M_3	M_4	M_5
J_1	O_{11}	2	6	5	3	4
	O_{12}	—	8	—	4	—
J_2	O_{21}	3	—	6	—	5
	O_{22}	4	6	5	—	—
	O_{23}	—	7	11	5	8
J_3	O_{31}	4	—	3	—	4
	O_{32}	—	4	3	5	2

3. THE NOVEL INITIALIZATION METHOD

In this part, we present the initialization method, it contains two sub-methods: Global Selection and Local Selection which can be used separately or jointly. The two sub-methods can allow us to assign each operation to the suitable machine by taking into account the occupation time and workload of machines so as to generate high-quality initial population.

3.1 Global Selection (GS)

This sub-method records all the occupation time of each selected machine from beginning to the end of all operations of all jobs, and find the minimum occupation time in a global view. Detailed steps are as follows:

1) Create a array (we call it time array) to record the occupation time of every machine, the length is the number of the machines, and set each element's value to 0;

2) Select a job randomly and insure the job to be selected only once, and then select the first operation of the selected job;

3) Find out the available machines of the operation, and then add the corresponding occupation time to the time array to form a second array (we call it temporary array);

4) Compare each element of the temporary array to find the shortest time, and then select the index of the available machine's corresponding time. If there is the same time among different machines in the temporary array, select a machine from the set of the same time machines randomly;

5) Add the current selected machine's occupation time to the time array so as to update the time array;

6) Select the next operation of the current job, and execute Step3 to Step5 until all the operations of the current job are selected orderly;

7) Go to Step 2 until all jobs are selected once.

Taking Table 1 as an example, we give the implementing process of GS. And the sketch map is presented in Fig.1. Create the time array first, set its length to be 5 according to the number of the machines, so the time array can be denoted by [0 0 0 0 0], then select a job randomly, we assume that the selected sequence of the jobs is J_3, J_1, J_2. The available machines of O_{31} are M_1, M_3, M_5, adding the corresponding occupation time of the machines [4 - 3 - 4] to the time array ("-" denotes the occupation time of the unavailable machines, and the time is infinity), we get temporary array [4 - 3 - 4], and we can select the shortest time 3 from the temporary array easily, so the selected machine of O_{31} is M_3, then add the selected machine's occupation time 3 to the time array [0 0 0 0 0], we get the updated time array [0 0 3 0 0]. In the next, for O_{32}, the time array is [0 0 3 0 0], and its available machines are M_2, M_3, M_4, M_5, add the corresponding occupation time of the machines [- 4 3 5 2] to the time array [0 0 3 0 0], we get the temporary array [- 4 6 5 2], so the selected machine of O_{32} is M_5, and the updated time array is [0 0 3 0 2].

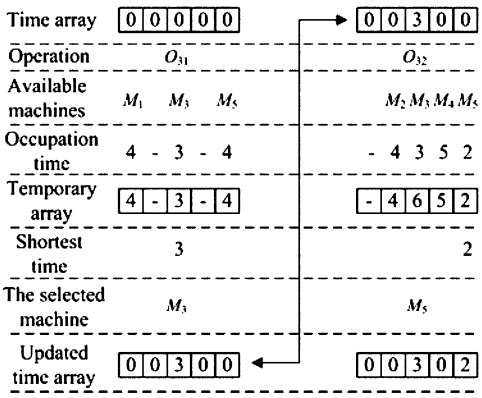


Fig.1. The process of assigning J_3 by GS

Then select the operation O_{11} of J_1, and the available machines of O_{11} are M_1, M_2, M_3, M_4, M_5, add the corresponding occupation time of the machines [2 6 5 3 4] to the time array [0 0 3 0 2] to generate the temperate array [2 6 8 3 6], so we select M_1 to execute O_{11}, then

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 22,2026 at 14:13:38 UTC from IEEE Xplore. Restrictions apply.

add the selected machine's occupation time 2 to the time array [0 0 3 0 2], and the updated time array changes into [2 0 3 0 2]. In GS, we do the above process until all jobs are selected once. The total process of GS is showed in Fig.2. So we can easily get the generated initial population [M_3 M_5 M_1 M_4 M_1 M_2 M_4].

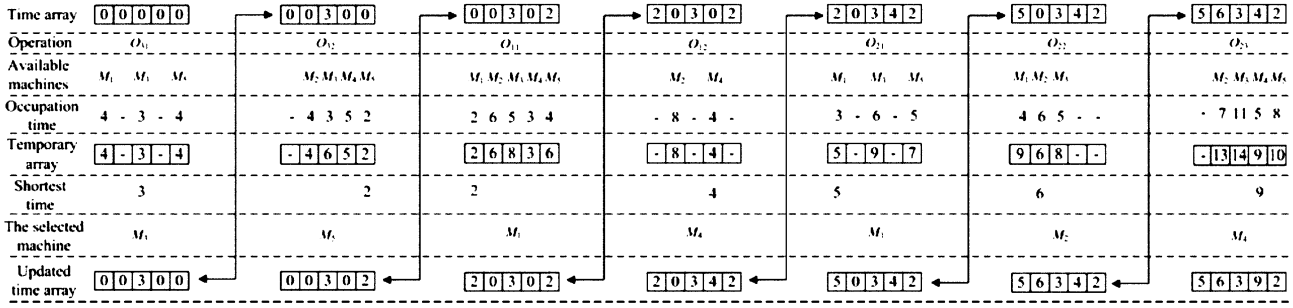


Fig.2. The process of solving Table 1 by GS

3.1 Local Selection (LS)

This sub-method records all the occupation time of each selected machine from beginning to the end of all operations of each job, and finds the minimum occupation time in a local view. Detailed steps are as follows:

1) Create a array (we call it time array) to record the occupation time of every machine, the length is the number of the machines, and set each element's value to 0;

2) Select a job randomly and insure the job to be selected only once, then select the first operation of the selected job;

3) Find out the available machines of the operation, and then add the corresponding occupation time to the time array to generate a second array (we call it temporary array);

4) Compare each element of the temporary array to find the shortest time, and then select the index of the available machine's corresponding time. If there is the same time among different machines in the temporary array, select a machine from the set of the same time machines randomly;

5) Add the current selected machine's occupation time to the time array so as to update the time array;

6) Select the next operation of the current job, and execute Step3 to Step5 until all the operations of the current job are selected;

7) Set each element of the time array to 0;

8) Go to Step 2 until all jobs are selected once.

We also give the implementing process of LS by taking the Table 1 as an example. Create the time array [0 0 0 0 0] first, then select a job randomly, we assume that the selected sequence of the jobs is also J_3, J_1, J_2 in order to make a clear comparison between GS and LS. The execution of J_3 in LS is the same as in GS, and the updated time array is [0 0 3 0 2].

In the next, when J_1 is being executed, the time array should be reset to [0 0 0 0 0], then add the corresponding occupation time of the available machines of O_{11} [2 6 5 3 4] to the time array [0 0 0 0 0], we get the temperate array [2 6 5 3 4], so we select M_1 to execute O_{11}.An illustrative figure is shown in Fig.3.

Following the same method, we make all jobs selected once and the sketch map is showed in Fig.4, so we can easily get the generated initial population [M_3 M_5 M_1 M_4 M_1 M_3 M_2].

The difference between GS and LS is the updating style of time array. In GS, the time array records all the occupation time of all jobs, however, the time array in LS records all the occupation time of each job. Because of the difference, GS can assign all jobs on rational machines considering the workload, LS can assign all operations of one job on rational machines considering the occupation time. When doing the manipulation of GA, we often use GS, LS and random jointly (GS+LS+Random) to generate initial population in order to enhance randomicity and the diversity of population. For example, 60% of initial population could be generated by GS, 30% by LS, and 10% by random.

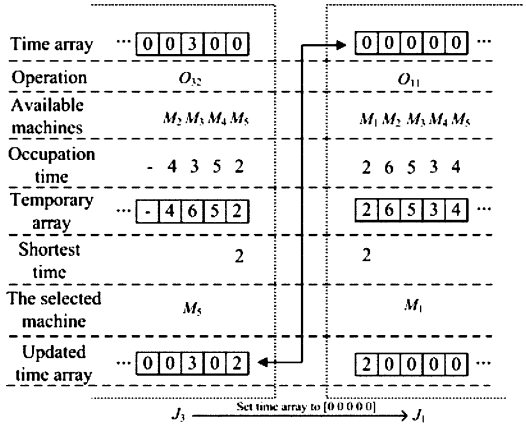


Fig.3. The process of assigning J_3 and J_1 by LS

4. NUMERICAL EXAMPLES

In order to test the proposed initialization method, we run GA initiated by our initialization method (we call the algorithm GA+GLR) on ten problems taken from

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 22,2026 at 14:13:38 UTC from IEEE Xplore. Restrictions apply.

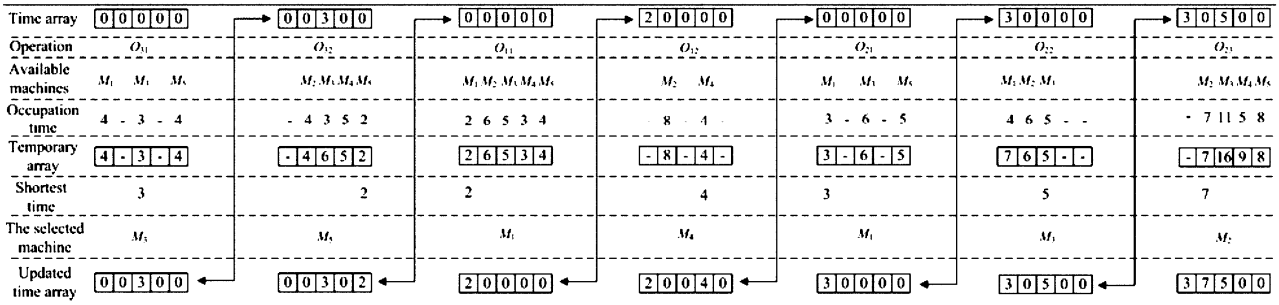


Fig.4. The process of solving Table 1 by LS

Brandimarte [9] in C++ on a Pentium IV running at 1.8 GHz, the flow chart of GA+GLR is presented in Fig.5.

Every problem will be tested five times and the parameters used in GA+GLR are chosen experimentally in order to get a satisfactory solution in an acceptable timespan. According to the complexity of the problems, the population size of GA+GLR ranges from 50 to 300, and through experimentation, the other parameter values are chosen as follows:

number of generation: 100
rate of initial assignments by GS: 0.6
rate of initial assignments by LS: 0.3
rate of initial assignments by random: 0.1
crossover probability: 0.7
mutation probability: 0.01

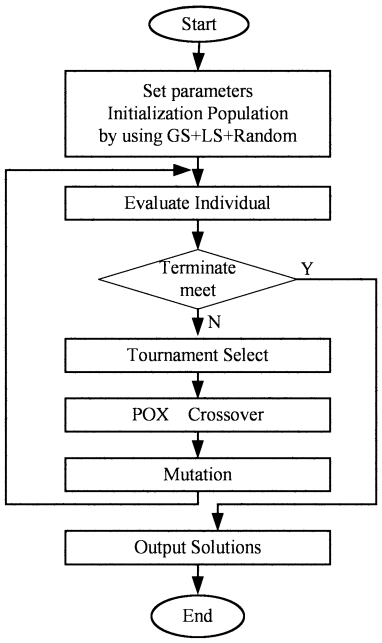


Fig.5. The process of GA+GLR

Mk04 test problem with 15 jobs and 8 machines from the ten problems is selected randomly to test GS+LS+Random in the aspect of generating high quality population. The average of best makespan and average of makspan are selected to evaluate the quality of initialization population. The experimental data is shown in table 2 and Fig.6.

Table 2 The quality of initialization population by two methods

Pop-ulation size	GS+LS+Random		Random	
	Average of best makespan	Average of makespan	Average of best makespan	Average of makespan
50	75.0	91.6	93.6	116.0
100	73.2	92.8	88.8	116.5
150	73.0	92.8	88.4	115.9
200	73.4	92.2	87.6	115.9
250	73.2	92.5	88.2	116.3
300	72.8	92.8	87.0	116.3
350	72.8	92.4	87.0	115.5
400	73.8	92.6	87.8	115.5
450	71.4	92.5	86.4	115.2
500	70.8	92.4	86.8	115.5

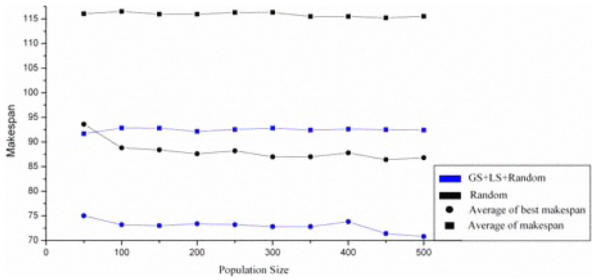


Fig.6. The comparison of two initialization methods

From the comparison between GS+LS+Random and Random method in Fig.6, we can see the quality of initialization population improved obviously with the increase of population size by the proposed initialization method.

In Fig.7, we draw the decrease of the average best makespan over five runs for the Mk04 initialized by GS+LS+Random and the common used random method respectively. It is possible to note that our algorithm improves the average best makespan very fast, and the best makespan (60) is reached after 40 generations. So we could see initialization method could affect the convergence speed and the quality of solutions from the comparison in Fig.7.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 22,2026 at 14:13:38 UTC from IEEE Xplore. Restrictions apply.

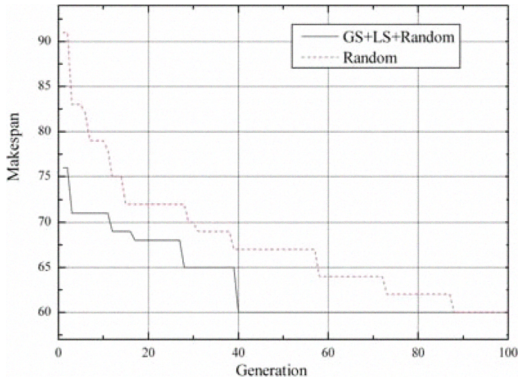


Fig.7. Decreasing of the makespan (Mk04)

We compare GA+GLR with other algorithms to verify the computational capability of it. The results are presented in Table 3. In the table, (LB, UB) denotes the optimum makespan if known, otherwise, the best lower and upper bound found to date. *Flex.* denotes the average number of equivalent machines per operation. P_s denotes the population size. AV(C_M) stands for the average makespan out of five runs. C_M^B denotes makespan of the best solution found by our proposed algorithm over five runs. M&G is the approach proposed by Mastrolilli *et al.* [3], and GENACE is proposed by Ho *et al.* [10].

Table 3 The comparison of computational results

Problem	$n\times m$	*Flex.*	(LB,UB)	M&G		GENACE			GA+GLR			
				C_M	AV(C_M)	P_s	C_M	AV(CPU)	P_s	C_M^B	AV(C_M)	AV(CPU)
Mk01	10×6	2.09	(36,42)	40	40.0	200	40	3.2	100	40	40	1.6
Mk02	10×6	4.01	(24,32)	26	26.0	200	32	3.6	300	26	26	2.6
Mk03	15×8	3.01	(204,211)	204	204.0	N/A	N/A	N/A	50	204	204	1.3
Mk04	15×8	1.91	(48,81)	60	60.0	200	67	6.1	100	60	60	6.2
Mk05	15×4	1.71	(168,186)	173	173.0	200	176	7.2	200	173	174	7.3
Mk06	10×15	3.27	(33,86)	58	58.4	200	67	10.7	200	58	59	15.7
Mk07	20×5	2.83	(133,157)	144	147.0	200	147	6.9	200	144	145	17.3
Mk08	20×10	1.43	(523)	523	523.0	200	523	19.1	50	523	523	2.2
Mk09	20×10	2.53	(299,369)	307	307.0	200	320	20.4	300	307	307	30.2
Mk10	20×15	2.98	(165,296)	198	199.2	200	229	27.6	200	201	202	32.5

Among the 10 test problems, Mk03 and Mk08 could get the optimal solution in the first generation by using GA+GLR. 8 problems out of the 10 problems could gain the same good results as M&G. Compared with GENACE, GA+GLR could get better solutions of 8 problems with short computational time.

5. CONCLUSION

In this paper, we propose a novel efficient initialization method (GS and LS) used in the initialization of GA manipulation to solve Flexible Job-shop Scheduling Problem. The comparison between GA+GLR and other algorithms proves that the proposed method can get acceptable and satisfactory solutions under the capacity constraints in reasonable time, so the initialization method is effective and efficient. Initialization is an crucial stage for algorithm, it can affect the quality of the solution, the efficiency of the algorithm and the computational time, so the initialization method is an important research direction, and another objective of this paper is to gain more insights into the research on the initialization method.

Based on the proposed initialization method, the future research interest will focus on:

1) how to adjust the proportion of the GS, LS and Random used in the initialization in order to generate better results;

2) use the idea of GS and LS in every iterative process of the algorithm's manipulation in order to further enhance the efficiency and shorten the computational time;

3) the hybrid usage of GS and LS with other algorithms.

ACKONWLEDGEMENT

This project is supported by 863 High Technology Plan Foundation of China under Grant No. 2006AA04Z131, 973 National Basic Research Program of China under Grant No.2004CB719405 and Program for New Century Excellent Talents in University under Grant No. NCET-08-0232. We also sincerely thank the anonymous referees for their valuable comments and suggestions, which have significantly improved the early version of the paper.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 22,2026 at 14:13:38 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] H.X. Chen, J. Ihlow, C. Lehmann. "A genetic algorithm for flexible jobshop scheduling", *In: IEEE International conference on Robotics and Automation, Detroit*, 2, pp1120-1125, 1999.

[2] L. Gao, C.Y. Peng, C. Zhou and P.G. Li. "Solving Flexible Job-shop Scheduling Problem Using General Particle Swarm Optimization", *Proceedings of The 36th International Conference on Computers and Industrial Engineering, Taipei, China,* pp3018–3027 , 2006.

[3] M. Mastrolilli, L.M. Gambardella. "Effective neighborhood functions for the flexible job shop problem", *Journal of Scheduling*, Vol. 3, pp3-20, 1996.

[4] NM. Najid, S. Dauzere-Peres, A. Zaidat. "A modified simulated annealing method for flexible job shop scheduling problem", *IEEE International conference on Systems, Man and Cybernetics*, Vol. 5, pp6-12, 2002.

[5] F. Pezzella, G. Morganti, G. Ciaschetti. "A genetic algorithm for the flexible job-shop scheduling problem", *Computers and Operations Research*, Vol. 35, No. 10, pp3202-3212, 2008.

[6] R. Shahryar, R. Hamid Tizhoosh, M.A. Magdy Salama. "A novel population initialization method for accelerating evolutionary algorithms", *Computers and Mathematics with Applications*, Vol. 53, No. 10, pp1605–1614, 2007.

[7] R.G Kang, C.Y Jung. "The Improved Initialization Method of Genetic Algorithm for Solving the Optimization Problem", *Neural Information Processing*, Vol. 4234, pp789-796, 2006.

[8] I. Kacem, S. Hammadi, P. Borne. "Approach by localization and multiobjective evolutionary optimization for flexible job-shop scheduling problems", *IEEE Transactions on Systems, Man and Cybernetics*, Vol. 32, No. 1, pp1-13, 2002.

[9] P. Brandimarte. "Routing and scheduling in a flexible job shop by tabu search." *Annals of Operations Research*; Vol. 41, pp157-183, 1993.

[10] NB Ho, JC Tay, M. K. Lai Edmund. "An effective architecture for learning and evolving flexible job shop schedules", *European Journal of Operational Research*, Vol. 179: pp316-333,2007.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 22,2026 at 14:13:38 UTC from IEEE Xplore. Restrictions apply.